

AWS Architecture Project Report

EduStream Cloud – Scalable and Highly Available Education Platform on AWS

1. Project Title

EduStream Cloud: Designing and Implementing a Scalable, Highly Available, Secure, and Serverless-Enabled Education Platform on AWS

2. Introduction

Cloud computing has become a core foundation for modern digital learning platforms because it enables scalability, flexibility, cost optimization, and high availability. Educational platforms need to support large numbers of students and instructors, host digital learning content, manage course information, and remain available even during peak usage such as assignment deadlines, examinations, and live course launches.

This project presents the architecture and implementation of **EduStream Cloud**, an education-domain application built on **Amazon Web Services (AWS)**. The platform is designed to support core educational workflows such as managing students and courses, storing learning content, and automating file-upload events. The implementation follows the **AWS Well-Architected Framework** principles and demonstrates the use of key AWS services including **VPC, EC2, IAM, RDS, S3, Application Load Balancer, Auto Scaling, CloudWatch, and Lambda**.

The project was implemented phase by phase on the AWS Console and includes both traditional infrastructure services and serverless event-driven automation.

3. Business Objective

The business objective of EduStream Cloud is to provide a secure, scalable, and highly available education platform capable of:

- managing student and course information,
- hosting educational content such as notes and files,
- supporting application growth with load balancing and auto scaling,
- protecting data using secure storage and controlled access,
- monitoring infrastructure and application health,
- automating backend file processing using serverless services.

The platform is intended to represent a realistic cloud-native education system that can be expanded into a full online learning portal.

4. Problem Statement

Traditional education platforms hosted on a single server often face the following problems:

- single point of failure,
- limited scalability under growing student traffic,
- weak separation between web, application, and database layers,
- poor monitoring and alerting,
- difficulty handling uploaded content reliably,
- manual operational overhead for content processing and infrastructure management.

To solve these issues, EduStream Cloud was designed as a multi-tier AWS-based architecture with high availability, monitoring, managed storage, and serverless event handling.

5. Project Scope

The scope of this project includes the design and implementation of a cloud-based education application architecture using AWS services. The project covers:

- network design using **Amazon VPC**,
- application hosting using **Amazon EC2**,
- relational data storage using **Amazon RDS MySQL**,
- object storage using **Amazon S3**,
- identity and permission management using **AWS IAM**,
- high availability using **Application Load Balancer** and **Auto Scaling Group**,
- monitoring using **Amazon CloudWatch**,
- serverless event-driven processing using **AWS Lambda** triggered by **S3 event notifications**.

The implementation also includes a working sample application called **EduStream**, built using **Python Flask**, that performs student management, course management, and S3 file upload.

6. Functional Requirements

The EduStream platform must satisfy the following functional requirements:

1. The system must allow the application to run on AWS infrastructure.
2. The platform must store student and course information in a relational database.
3. The platform must support uploading educational files to cloud storage.

4. The application must allow basic student management and course management operations.
 5. The platform must route user traffic through a load balancer.
 6. The infrastructure must support multiple EC2 instances for application availability.
 7. The platform must generate monitoring metrics and alarms for key services.
 8. The platform must trigger a serverless workflow whenever a course file is uploaded to S3.
 9. The Lambda function must capture upload event details and log them to CloudWatch.
-

7. Non-Functional Requirements

The EduStream platform must satisfy the following non-functional requirements:

7.1 Scalability

The system must scale to handle increasing user traffic and application load without requiring a complete redesign.

7.2 High Availability

The application should remain available even if one application instance fails.

7.3 Security

The system must use private database networking, security groups, IAM roles, and controlled access to resources.

7.4 Performance

The platform should provide acceptable response times for common operations such as viewing students, adding courses, and uploading files.

7.5 Reliability

Data should be stored durably and system failures should be minimized through managed AWS services and health monitoring.

7.6 Cost Optimization

The architecture should use cost-effective instance sizes and managed services appropriate for a practice or prototype environment.

7.7 Observability

The system should be monitored using metrics, alarms, and logs to detect failures and resource bottlenecks.

8. AWS Well-Architected Framework Alignment

This project was designed with reference to the **AWS Well-Architected Framework**, especially the following pillars:

8.1 Operational Excellence

- Infrastructure was implemented in clear phases.
- Monitoring was added using CloudWatch alarms and log groups.
- Application deployment was standardized on EC2 instances.
- Auto Scaling and load balancing reduce manual operational dependency.

8.2 Security

- IAM roles were used instead of embedding AWS access keys on EC2.
- S3 access was granted to EC2 through an IAM role.
- RDS was deployed in private subnets with no public access.
- Security groups were used to control traffic between ALB, EC2, and RDS.
- S3 bucket encryption and private access settings were enabled.

8.3 Reliability

- Application instances were placed behind an Application Load Balancer.
- Auto Scaling Group maintains multiple instances and replaces unhealthy instances.
- RDS provides managed database availability and backup options.
- CloudWatch alarms improve operational response to failures.

8.4 Performance Efficiency

- The application uses a lightweight Flask-based design.
- ALB distributes traffic across application instances.
- RDS provides managed relational database performance.
- S3 stores static and uploaded content efficiently outside the EC2 instance.

8.5 Cost Optimization

- Small instance types such as micro instances were selected for practice use.
- Managed services reduce operational overhead.
- S3 provides cost-effective storage for educational files.

- The architecture is modular, so resources can be scaled only when needed.

8.6 Sustainability

- Using managed and auto-scaled infrastructure helps avoid overprovisioning.
 - S3 lifecycle policies can be added for older backup data.
 - Auto Scaling allows resources to align more closely with actual demand.
-

9. Proposed Use Case in Education Domain

The selected use case for this project is a **cloud-based education platform** that can be used by a training institute, college, online learning startup, or digital coaching platform.

The system supports the following high-level educational workflow:

- administrators and instructors manage courses,
- student records are stored in a relational database,
- educational notes and course materials are uploaded to cloud storage,
- users access the application through a load-balanced endpoint,
- uploaded course content automatically triggers a serverless backend process for metadata logging and monitoring.

This makes EduStream a practical representation of a modern LMS-style education platform.

10. High-Level Solution Overview

EduStream Cloud uses a **multi-tier architecture**:

- **Presentation / Access Layer**: Application Load Balancer
- **Application Layer**: EC2 instances running a Flask application
- **Database Layer**: Amazon RDS MySQL in private DB subnets
- **Object Storage Layer**: Amazon S3 for course files and uploads
- **Monitoring Layer**: Amazon CloudWatch
- **Serverless Automation Layer**: AWS Lambda triggered by S3 events
- **Identity and Security Layer**: IAM roles and Security Groups
- **Scalability Layer**: Auto Scaling Group

This design improves separation of concerns and makes the application more production-like than a single-server deployment.

11. Architecture Components

11.1 Amazon VPC

A dedicated **Virtual Private Cloud (VPC)** was created to isolate the EduStream environment. The VPC provides logical network boundaries for application and database resources.

Purpose

- isolate cloud resources from other AWS environments,
 - create public and private subnets,
 - define routing and security boundaries.
-

11.2 Public and Private Subnets

The architecture uses multiple subnets to separate internet-facing and private components.

Public Subnets

Used for:

- Application Load Balancer
- EC2 application instances used for app access and scaling

Private DB Subnets

Used for:

- Amazon RDS MySQL database instance

Purpose

- keep the database private,
 - allow only the web/application tier to communicate with the database,
 - support high availability across multiple Availability Zones.
-

11.3 Internet Gateway

An Internet Gateway was attached to the VPC to allow public internet access for public subnet resources such as the ALB and application EC2 instances.

Purpose

- allow inbound and outbound internet traffic for public-facing components.
-

11.4 Route Tables

Route tables were configured to route public subnet traffic through the Internet Gateway and to keep database subnets private.

Purpose

- control subnet-level traffic flow,
 - enforce separation between public and private layers.
-

11.5 Security Groups

Security groups were created for each major layer:

EduStream-ALB-SG

Allows:

- HTTP/HTTPS traffic from the internet to the load balancer.

EduStream-App-SG

Allows:

- HTTP from the ALB security group,
- SSH from the administrator IP for management access.

EduStream-RDS-SG

Allows:

- MySQL port 3306 from the application security group only.

Purpose

- enforce least-privilege network access,
 - protect the database from public access,
 - ensure only approved service-to-service communication.
-

11.6 IAM Roles

IAM roles were used for controlled service access.

EC2 Role

The EC2 application instances were attached to an IAM role that allows:

- access to S3,
- CloudWatch integration,
- optional Systems Manager support.

Lambda Role

A Lambda execution role was created with permissions to:

- write logs to CloudWatch,
- read S3 object metadata.

Purpose

- avoid storing access keys on servers,
 - implement secure, temporary credential-based access to AWS services.
-

11.7 Amazon EC2

Amazon EC2 instances were used to host the EduStream application.

Application deployed

A **Python Flask** application was created and deployed on EC2. The application supports:

- student management,
- course management,
- file upload to S3.

Purpose

- provide the application compute layer,
 - host the education portal web application,
 - connect to both RDS and S3.
-

11.8 Amazon RDS MySQL

Amazon RDS MySQL was used as the relational database backend.

Database usage

The database stores structured application data such as:

- students,
- courses.

Purpose

- provide managed relational storage,
- remove the burden of self-managing MySQL on EC2,
- support backups, monitoring, and reliable database operations.

Database placement

The RDS instance was deployed in private DB subnets and was not publicly accessible.

11.9 Amazon S3

Amazon S3 was used to store educational content and uploaded files.

Bucket usage

The S3 bucket stores:

- course notes,
- uploaded educational files,
- future course assets and backups.

Example folder structure

- courses/videos/
- courses/notes/
- courses/thumbnails/
- assignments/
- submissions/
- backups/

Purpose

- durable and scalable object storage,
- offload uploaded files from the application server,

- provide a storage layer for education content.
-

11.10 Application Load Balancer (ALB)

An Application Load Balancer was created to distribute incoming user traffic across multiple EC2 instances.

Purpose

- provide a single public entry point,
 - improve availability,
 - route traffic to healthy instances,
 - support future HTTPS and path-based routing enhancements.
-

11.11 Auto Scaling Group (ASG)

An Auto Scaling Group was created to maintain multiple application instances.

Purpose

- automatically launch and maintain the desired number of EC2 instances,
 - improve high availability,
 - replace unhealthy instances,
 - provide the foundation for scaling based on traffic or CPU.
-

11.12 Launch Template

A Launch Template was created using an AMI generated from the working EduStream application server.

Purpose

- standardize future EC2 instance launches,
 - ensure all app instances use the same application configuration and software stack,
 - simplify Auto Scaling deployment.
-

11.13 Amazon CloudWatch

CloudWatch was used for monitoring, alarms, and logs.

Monitoring use cases

- EC2 CPU utilization alarm

- ALB unhealthy host alarm
- RDS CPU alarm
- RDS free storage alarm
- Lambda execution logs
- optional system log collection from EC2

Purpose

- operational visibility,
 - proactive issue detection,
 - infrastructure health monitoring.
-

11.14 AWS Lambda

A Lambda function was created to process S3 upload events.

Function behavior

When a file is uploaded into the configured S3 prefix, Lambda:

- receives the S3 event,
- extracts bucket and object details,
- reads metadata such as object size and content type,
- writes event details to CloudWatch Logs.

Purpose

- demonstrate serverless automation,
 - enable event-driven processing,
 - reduce the need for manual or server-based background tasks.
-

12. Detailed Architecture Design

12.1 Logical Flow of the Application

The end-to-end request and processing flow is as follows:

1. A user opens the EduStream application using the **Application Load Balancer DNS endpoint**.
2. The ALB forwards the request to one of the healthy EC2 application instances.
3. The Flask application running on EC2 serves the user request.
4. If the request involves student or course data, the EC2 instance queries **Amazon RDS MySQL**.
5. If the request involves file upload, the EC2 instance uploads the file to **Amazon S3** using its IAM role.
6. When a file is uploaded into the configured S3 prefix, **S3 Event Notification** triggers **AWS Lambda**.
7. Lambda reads object metadata and logs the event into **CloudWatch Logs**.

8. **CloudWatch** also monitors EC2, ALB, and RDS through metrics and alarms.

12.2 Layered Architecture View

User Access Layer

- Application Load Balancer

Compute / Application Layer

- EC2 instances in Auto Scaling Group
- Flask application deployed on each instance

Data Layer

- Amazon RDS MySQL in private subnets
- Amazon S3 for object storage

Security Layer

- IAM roles
- Security groups
- private RDS placement

Monitoring Layer

- CloudWatch metrics, alarms, and logs

Automation / Serverless Layer

- Lambda triggered by S3 upload events
-

13. Implementation Methodology

The project was implemented in **eight phases** to simulate a structured cloud deployment process.

14. Phase-Wise Implementation Summary

Phase 1 — Network Foundation (VPC and Subnets)

The first phase focused on creating the network base for the application.

Activities completed

- created a dedicated VPC for EduStream,
- created public subnets for ALB and app servers,
- created private DB subnets for RDS,
- configured route tables,
- attached an Internet Gateway,
- established the network layout for multi-tier deployment.

Outcome

A secure VPC architecture with public and private subnet separation was established.

Phase 2 — Security Foundation (IAM and Security Groups)

The second phase focused on identity and access control.

Activities completed

- created IAM role for EC2 with S3 access,
- configured security groups for ALB, application servers, and RDS,
- enforced traffic rules between architecture layers.

Outcome

The project gained a controlled and secure permission model for service-to-service access.

Phase 3 — Database Layer (Amazon RDS MySQL)

The third phase created the relational database backend.

Activities completed

- created DB subnet group using private DB subnets,
- launched RDS MySQL instance,
- configured RDS security group,
- disabled public access to the database,
- stored the RDS endpoint for application use.

Outcome

A private managed MySQL database was successfully deployed for application data.

Phase 4 — Storage Layer (Amazon S3)

The fourth phase implemented object storage for educational files.

Activities completed

- created an S3 bucket for course storage,
- enabled private bucket access and encryption,
- organized bucket prefixes for course notes, videos, submissions, and backups,
- verified IAM-based access design for EC2.

Outcome

Durable and scalable object storage was established for educational content.

Phase 5 — EC2 Application Server and EduStream Flask App

The fifth phase introduced the working application.

Activities completed

- launched an EC2 instance for the application server,
- installed Python, Flask, PyMySQL, and Boto3,
- created and deployed the EduStream Flask application,
- integrated the application with RDS and S3,
- tested student creation, course creation, and file upload.

Flask application features

- **Home page**
- **Students page** for adding and listing students
- **Courses page** for adding and listing courses
- **Upload page** for uploading files to S3

Outcome

A working education application was deployed on EC2 and integrated with RDS and S3.

Phase 6 — High Availability Layer (ALB + ASG)

The sixth phase upgraded the app into a highly available architecture.

Activities completed

- created an AMI from the working app server,

- created a Target Group,
- created an Application Load Balancer,
- created a Launch Template,
- created an Auto Scaling Group across subnets,
- validated target health and app access through ALB DNS,
- removed direct public HTTP exposure to app instances after ALB testing.

Outcome

The application was converted from a single-server deployment to a load-balanced, multi-instance architecture.

Phase 7 — Monitoring Layer (CloudWatch)

The seventh phase implemented operational monitoring.

Activities completed

- created EC2 CPU alarm,
- created ALB unhealthy host alarm,
- created RDS CPU alarm,
- created RDS low free storage alarm,
- optionally configured CloudWatch log collection.

Outcome

The platform gained observability and alerting capabilities for application and database health.

Phase 8 — Serverless Automation (Lambda + S3 Events)

The eighth phase added serverless event-driven processing.

Activities completed

- created a Lambda execution role,
- created Lambda function for S3 upload processing,
- configured S3 event notification on the bucket,
- uploaded test files to trigger the Lambda,
- verified CloudWatch logs for upload events.

Outcome

The architecture now includes a serverless workflow that automatically processes file upload events.

15. Application Design and Code Overview

A lightweight **Python Flask** application was built to demonstrate the application layer.

Core application features

15.1 Home Page

The home page acts as the main portal and provides navigation links to:

- Students
- Courses
- Upload

15.2 Students Module

This page allows:

- adding student records,
- listing student records stored in RDS.

15.3 Courses Module

This page allows:

- adding course records,
- listing course records stored in RDS.

15.4 Upload Module

This page allows:

- selecting a file from the browser,
- uploading the file to the S3 bucket,
- storing the file under the course notes prefix.

15.5 Database Initialization

The application creates the following database tables if they do not exist:

- `students`
 - `courses`
-

16. Database Design

The database used in the project is **MySQL on Amazon RDS**.

16.1 Students Table

Stores student information such as:

- student ID
- student name
- student email

16.2 Courses Table

Stores course information such as:

- course ID
- course title
- instructor name

16.3 Future Expandable Tables

The platform can be extended with:

- instructors
 - enrollments
 - assignments
 - submissions
 - quizzes
 - course progress
-

17. S3 Storage Design

The S3 bucket was structured to represent a realistic education-content storage layout.

Folder / Prefix Design

- `courses/videos/` → lecture videos
- `courses/notes/` → PDFs, notes, documents
- `courses/thumbnails/` → course thumbnails
- `assignments/` → assignment files
- `submissions/` → student submissions
- `backups/` → backup objects or exported content

Benefits

- clear separation of content types,
 - easier lifecycle policy design,
 - better operational organization.
-

18. Monitoring and Alerting Design

CloudWatch was used to monitor the health of the platform.

18.1 EC2 CPU Alarm

Purpose: Detect high CPU usage on the application tier.

18.2 ALB Unhealthy Host Alarm

Purpose: Detect when one or more application instances become unhealthy behind the load balancer.

18.3 RDS High CPU Alarm

Purpose: Identify database stress or heavy query load.

18.4 RDS Low Storage Alarm

Purpose: Detect storage exhaustion risk before the database becomes unavailable.

18.5 Lambda Logs

Purpose: Verify S3 upload event processing and troubleshoot serverless execution.

19. Serverless Event Automation Design

A Lambda function named **EduStream-S3-Processor** was integrated with S3.

Trigger condition

- file uploaded into the configured S3 prefix, such as `courses/notes/`

Lambda processing logic

- receives the event payload,
- extracts bucket name and object key,
- performs `head_object` lookup on the uploaded file,
- captures metadata such as content type and size,
- logs the result to CloudWatch.

Benefits

- event-driven architecture,
 - no need for always-running background workers,
 - extensible for future workflows such as notifications, metadata indexing, or virus scanning.
-

20. Security Design

Security was a major consideration in the architecture.

20.1 Network Security

- RDS was kept in private DB subnets.
- Security groups restricted traffic to only required ports.
- ALB handled public access while EC2 instances were protected behind SG rules.

20.2 Identity and Access Management

- EC2 accessed S3 through an IAM role.
- Lambda used a dedicated execution role.
- Direct credential embedding for AWS API access was avoided.

20.3 Storage Security

- S3 bucket was configured with private access.
- Bucket encryption was enabled.
- Versioning could be enabled for object protection and recovery.

20.4 Database Security

- Database access was limited to the application security group.
- Public database exposure was disabled.

20.5 Future Security Improvements

- move DB credentials to **AWS Secrets Manager**,

- use **HTTPS** on the ALB with ACM certificates,
 - enable **AWS WAF** for web application protection,
 - use **KMS customer-managed keys** for stronger encryption controls.
-

21. Scalability and High Availability Design

21.1 Application Layer Scalability

The Auto Scaling Group allows the application tier to run multiple instances and scale horizontally.

21.2 Traffic Distribution

The Application Load Balancer distributes incoming requests across healthy instances.

21.3 Database Layer

RDS provides a managed database layer and can be extended to Multi-AZ deployment for stronger availability.

21.4 Object Storage Scalability

S3 scales automatically and can store very large amounts of educational content without infrastructure management.

21.5 Event Processing Scalability

Lambda scales automatically in response to S3 events and does not require pre-provisioned servers.

22. Cost Optimization Considerations

The project was built with cost-conscious decisions suitable for a practice implementation.

Cost-saving design choices

- use of micro EC2 and RDS instance sizes where possible,
- use of managed services instead of self-managed servers,
- storage of static content in S3 instead of on EC2,
- use of Lambda for event processing instead of dedicated worker servers,
- use of CloudWatch alarms for targeted monitoring rather than complex third-party tooling.

Potential future cost optimizations

- move older S3 objects using lifecycle policies,
 - use Auto Scaling policies to reduce idle instances in low traffic periods,
 - evaluate RDS instance sizing after workload analysis,
 - use AWS Cost Explorer and Budgets for cost monitoring.
-

23. Testing and Validation Performed

The EduStream platform was tested at multiple levels.

23.1 Application Testing

- accessed the home page through EC2 and then through the ALB,
- added student records successfully,
- added course records successfully,
- uploaded files to S3 successfully.

23.2 Database Testing

- verified RDS connectivity from EC2,
- confirmed creation and retrieval of student and course data.

23.3 S3 Testing

- confirmed file uploads from the Flask application to the configured S3 bucket and prefix.

23.4 Load Balancer Testing

- verified the application opened successfully using the ALB DNS name,
- validated target group health checks.

23.5 Monitoring Testing

- confirmed CloudWatch alarms were created for EC2, ALB, and RDS.

23.6 Lambda Testing

- uploaded a file into the S3 bucket,
 - verified that S3 triggered Lambda,
 - confirmed event logs appeared in CloudWatch Logs.
-

24. Advantages of the Implemented Architecture

The EduStream AWS architecture offers several benefits:

1. **Scalable application layer** using Auto Scaling and load balancing.
 2. **High availability** through multiple EC2 instances and ALB health checks.
 3. **Secure database deployment** in private subnets.
 4. **Durable storage** for course content using S3.
 5. **Serverless automation** using Lambda and S3 events.
 6. **Monitoring and alerting** using CloudWatch.
 7. **Modular architecture** that can be expanded with more services in the future.
 8. **Alignment with AWS best practices** and the Well-Architected Framework.
-

25. Limitations of the Current Implementation

Although the project successfully demonstrates a strong AWS architecture, there are still areas that could be improved for a production-grade deployment.

Current limitations

- database credentials are stored in the application configuration instead of Secrets Manager,
 - HTTPS is not yet enforced on the ALB,
 - the application is a prototype Flask app and not a full-featured LMS,
 - no CI/CD pipeline is configured,
 - the database is not yet configured for Multi-AZ failover,
 - advanced application logging and tracing are limited,
 - there is no CDN layer for content delivery.
-

26. Future Enhancements

The EduStream platform can be extended significantly.

26.1 Security Enhancements

- integrate **AWS Secrets Manager** for DB credentials,
- use **AWS Certificate Manager** and HTTPS on ALB,
- add **AWS WAF** for web protection,
- use more restrictive bucket and IAM policies.

26.2 Availability Enhancements

- enable **RDS Multi-AZ**,
- move application EC2 instances to private app subnets behind NAT,
- use health-based scaling policies.

26.3 Performance Enhancements

- add **Amazon CloudFront** for faster content delivery,
- use **ElastiCache** for caching course data or sessions,
- optimize DB queries and indexes.

26.4 Application Enhancements

- add instructor and student login functionality,
- build assignment submission workflows,
- add video streaming and course progress tracking,
- implement quizzes, grading, and analytics dashboards.

26.5 DevOps Enhancements

- create a CI/CD pipeline using **CodePipeline**, **CodeBuild**, and **CodeDeploy**,
- use Infrastructure as Code with **AWS CloudFormation** or **Terraform**,
- add automated backup and retention policies.

26.6 Analytics Enhancements

- send application and usage logs to **Amazon Athena**, **QuickSight**, or a data lake for reporting and analytics.

27. Draw.io Architecture Diagram Guidance

The architecture diagram for this project should visually represent the AWS services and their relationships. The recommended diagram layout is:

Top Layer – Users / Internet

- User / Student / Instructor icon
- Arrow to **Application Load Balancer**

Network Boundary – VPC

Create a large VPC boundary labeled **EduStream-VPC**

Inside the VPC, show two public subnets and two private DB subnets.

Public Subnet 1

- EC2 App Instance 1
- optionally show ASG grouping boundary

Public Subnet 2

- EC2 App Instance 2
- ASG / scalable app tier

ALB Placement

Place the **Application Load Balancer** across the public subnets.

Private DB Subnets

- Amazon RDS MySQL in DB subnet group

S3 Bucket

Place Amazon S3 outside the VPC boundary or adjacent to the application layer, connected from the EC2 application tier.

Lambda

Place Lambda connected from the S3 bucket via **S3 Event Notification**.

CloudWatch

Place CloudWatch connected to:

- EC2 / ASG
- ALB
- RDS
- Lambda

IAM

Add IAM role icons associated with:

- EC2 role
- Lambda role

Key arrows to include

- User → ALB
- ALB → EC2 instances
- EC2 → RDS
- EC2 → S3
- S3 → Lambda
- Lambda → CloudWatch Logs
- EC2 / ALB / RDS → CloudWatch

28. AWS Services Used and Purpose Table

AWS Service	Purpose in EduStream
Amazon VPC	Isolates the EduStream cloud network and hosts all core infrastructure
Public Subnets	Host internet-facing application resources such as ALB and EC2 instances
Private DB Subnets	Host the RDS database securely without public exposure
Internet Gateway	Enables internet connectivity for public resources
Route Tables	Control routing for public and private subnets
Security Groups	Restrict network traffic between ALB, EC2, and RDS
IAM	Provides secure role-based access for EC2 and Lambda
Amazon EC2	Hosts the EduStream Flask application
Amazon RDS MySQL	Stores structured data such as students and courses
Amazon S3	Stores course notes, files, assets, and backups
Application Load Balancer	Distributes traffic across healthy application instances
Auto Scaling Group	Maintains and scales EC2 instances for high availability
Launch Template	Standardizes instance launch configuration for ASG
Amazon CloudWatch	Provides metrics, alarms, and logs for monitoring
AWS Lambda	Processes S3 upload events in a serverless manner
S3 Event Notification	Triggers Lambda when a file is uploaded

29. Learning Outcomes from the Project

This project provided practical understanding of how to design and implement a cloud-native architecture on AWS. The key learning outcomes include:

- designing a multi-tier AWS architecture from scratch,
 - understanding VPC, subnets, route tables, and security groups,
 - deploying and connecting EC2, RDS, and S3,
 - implementing high availability with ALB and Auto Scaling,
 - using CloudWatch for monitoring and alarms,
 - building event-driven automation with Lambda and S3,
 - aligning technical implementation with business and architectural requirements.
-

30. Conclusion

The **EduStream Cloud** project successfully demonstrates the design and implementation of a scalable, secure, highly available, and serverless-enabled education platform on AWS. The architecture uses AWS managed services to separate networking, compute, storage, monitoring, and automation concerns into a clean multi-tier design.

The project began with a secure VPC foundation, added a relational database and object storage, deployed a working education web application on EC2, introduced high availability through an Application Load Balancer and Auto Scaling Group, implemented operational monitoring through CloudWatch, and completed the serverless automation layer using Lambda triggered by S3 events.

This implementation shows how AWS services can be combined to build a practical education-domain solution that is not only functional but also aligned with cloud architecture best practices. With further enhancements such as HTTPS, Secrets Manager, CI/CD, and advanced LMS features, EduStream can evolve into a more production-grade learning platform.

31. References

1. AWS Well-Architected Framework
 2. Amazon EC2 Documentation
 3. Amazon RDS Documentation
 4. Amazon S3 Documentation
 5. Elastic Load Balancing Documentation
 6. Auto Scaling Documentation
 7. Amazon CloudWatch Documentation
 8. AWS Lambda Documentation
 9. AWS IAM Documentation
-